

AGENTIC INTELLIGENCE & REPORTING LAYER FOR ENTERPRISE MOBILITY

Shift Readiness Agent

A line manager at a 24/7 support centre has one question at ten to nine: **is my floor staffed at nine, and if not, what do I do about it?** This is the agent that answers it.

PERSONA

Team / line
manager

DATASET

Anonymised trip
logs · 2.3M rows

SLICE

Clearwater Campus ·
09:00 · 11 Jun 2026

FORMS

Alerting · Narrative ·
Chat · Comms

08:50 on a Thursday. Twenty-four people due in ten minutes.

The manager is accountable for the floor. The data that predicts the floor sits in the transport system, in a format built for someone else.

WHAT HAPPENS TODAY

Assemble, then guess

- Open the transport portal — a list of cabs and delay flags.
- Cross-check the roster in a spreadsheet.
- Ring three or four people who might be stuck.
- Guess how short the queue will be.
- Find out you were wrong at 09:10, from the call queue.

The signal is rich. The insight is missing. The actions are manual.

WHAT THE AGENT DOES

Answer, priced, with the message written

- Watches every rostered agent's commute against the clock.
- Projects the floor and the service level per queue.
- Opens an alert when a threshold breaks — before nine, not after.
- Names who can cover, from people verifiably already on the floor.
- Hands back a message the manager can forward unedited.

Senses. Reasons. Acts. Without being asked.

The brief names three personas. We chose the one that nobody builds for — and the one where a twenty-minute delay turns into money in under an hour.

What MoveInSync asked for

"A metric without context is just a number. **OTA is 78%** matters far less than **it was 85% last month**, **SLA is 90%**, and **two vendors are responsible for the gap.**"

Problem statement, §4 — Problem definition

"An agentic layer that **senses** what's happening, **reasons** about what it means, and **acts** — with minimal human prompting."

Problem statement, §4 — Desired improvement

The operative words are **minimal human prompting**. A dashboard waits to be looked at. A chatbot waits to be asked. Neither of those is what was requested.

MANDATORY REQUIREMENTS → OUR ANSWER

Working, demo-able prototype on the provided dataset	Live board on 2.3M loaded rows. 112 tests.
Agentic — senses, reasons, acts	Clock → deterministic reasoning → alert, draft, recorded action.
Serves one named persona	Team / line manager. Built only for them.
Contextualises against ≥1 reference point	Five: team norm, weekday, recent trend, peer sites, structural slack.
Good-to-have: ≥2 solution forms	Five: alerting, narrative, chat, decision support, comms.
Good-to-have: handles messy data	Nine documented quirks, each counted at load.
Bonus: deployability into the platform	Tenant-scoped, ~15 ms/tick, one seam to swap.

Three personas. We chose the one nobody builds for.

The first two operate the transport service. The third only lives with it — which is exactly why their tooling is missing.

NOT THIS BUILD

Transport manager

Owns vendor coordination, escalations, shift planning. Operates the service.

Well served by existing ops dashboards. We surface a vendor pattern *to* them, as one escalation option.

NOT THIS BUILD

Transport & facilities head

Owns budget, SLA accountability, leadership reporting. Strategic, monthly cadence.

Our alert output is already forwardable upward without rework — but their own view is out of scope.

OUR PERSONA

Team / line manager

Shift-based ops. Needs to know who made it, who was late, and how delays ripple into floor readiness.

- Cannot call a vendor.
- Cannot re-plan a route.
- Can only move people on their own floor.

Why this constraint made the product better. Because the manager can only act on their own floor, every recommendation had to be executable within ten minutes by one person. No "contact the vendor" hand-waving. That is what forced us to compute cover candidates who are **verifiably already at their desks**.

One row in the trip log rewrote the whole design

Ride_data_trip-June_2026.csv · one leg, Clearwater Campus, 09:00 shift

SHIFT_TYPE	PLANNED_DROP	ACTUAL_DROP	MINUTES PAST BELL	DELAY_REASON
09:00	08:55	09:20	+20	NODELAY

THE DISAGREEMENT

Transport measures the trip. The cab ran within tolerance, so the trip is clean and the operator's dashboard is green.

The manager lives with the floor. The desk was empty for twenty minutes and the call queue felt every second of it.

70% of arrivals that miss this shift's start are stamped NODELAY.

THE DESIGN RULE IT PRODUCED

The product **never reads** `delay_reason`. Not once, anywhere in the codebase.

It derives lateness from the only thing the manager cares about: was a body in a chair when the shift bell rang.

The hard alert trigger is the deterministic "**planned drop has passed, no arrival recorded**" — which has no false positives, with or without GPS.

Four findings from three months of trip logs

Read before a line of product code was written. Each one changed the design.

51%

of riders hold one fixed shift all month

A team is **derivable**, not invented. Our 24-person roster and 24-person cover pool are real riders who took this shift on 40+ days.

5_{min} vs 13_{min}

Planned buffer, against journey-time noise

The plan sits *inside its own error bar*. Half of all arrivals are late **by construction**. Escalating daily cannot fix a schedule built that way — so we report it as a standing benchmark.

46% vs 29%

Clearwater is the worst site on this shift

Against four peer sites. Cedar Ridge runs the same 09:00 shift at **under 1%**. This is a site problem, not a law of physics — and the peer number is one of the five benchmarks in every alert.

70%

of late-for-shift arrivals are stamped **NODELAY**

The operator's own metric cannot see this problem. That is the gap the product lives in.

A metric, contextualised, is still not a decision

Three rungs. Most tools stop at the first. The brief asks for the third.

RUNG 1 · A NUMBER

What a dashboard shows

Coverage 67%

True, timely, and useless. Sixty-seven percent of *what*, costing *whom*, fixed *how*?

The manager still has to do all the thinking.

RUNG 2 · WITH CONTEXT

What good analytics shows

Coverage 67%, against a 75% floor.
Tracking toward 4 late — about the usual 4 for this team. Clearwater runs 46% late against 29% across four peer sites.

Now it means something. The manager still has to decide and still has to write the message.

RUNG 3 · A DECISION

What the agent delivers

Billing is **4 short until 09:19**. Service level **15%** now; the day's 80% target still holds.
Move Agent 27, 29, 35, 31 — already on the floor. Otherwise 4 night agents are held 19 min and miss the 09:15 cab home. **Message drafted. One click sends it.**

Effort removed, not relocated.

The board at 08:55 — the worst minute of the morning

Clearwater Campus 09:00 shift, 2026-06-11

08:55

3 of 24 on the floor

16 expected by 09:05, 67% of the roster, 2 queues below service level

Resume

Jump to 08:55

Start over

30x

60x

180x

Billing Support

Short

1 of 12 on the floor

0 on the way 11 at risk 8 expected by 09:05

Service level now

15%

At full strength

92%

Day so far

92% holds 80%

Needs on the queue

11

Technical Support

Short

2 of 12 on the floor

0 on the way 10 at risk 8 expected by 09:05

Service level now

22%

At full strength

92%

Day so far

92% holds 80%

Needs on the queue

11

Everyone rostered for 09:00

Agent	Queue	Where they are	Arrives	Cab
Agent 22	Technical Support	● On the floor	08:48	Arjun Mikhailov
Agent 01	Billing Support	● On the floor	08:53	Karan Mikhailov
Agent 18	Technical Support	● On the floor	08:53	Amit Mikhailov
Agent 02	Technical Support	● Aboard	08:55 ±17	Divya Sokolov
Agent 13	Billing Support	● Aboard	08:56 ±17	Rahul Morozov
Agent 23	Billing Support	● Aboard	08:57 ±17	Amit Mikhailov
Agent 16	Technical Support	● Aboard	08:57 ±18	Rahul Mikhailov
Agent 08	Technical Support	● Aboard	08:57 ±19	Pooja Mikhailov
Agent 11	Billing Support	● Aboard	08:58 ±18	Amit Mikhailov
Agent 05	Billing Support	● Aboard	09:00 ±18	Divya Kozlov
Agent 07	Billing Support	● Aboard	09:02 ±18	Karan Mikhailov

Technical Support

since 08:01

4 short until 09:29, service level 22%

Technical Support is 4 short and 67% covered for 09:00. Service level 22% now against 80%; the day still holds. Recommended: move the early shift onto the queue with Agent 28, Agent 36, Agent 38, Agent 40. If nobody acts, 4 night agents held 29 min past shift end, 4 would miss the 09:15 cab home.

tracking towards 4 late, about the usual 4 for this team

Thu runs 50% late at this site against 32% on Fri

the last 3 Thus ran 44% late for this team

What you can do

Keep the night shift on

Positions stay manned, but 4 night agents held 29 min past shift end, 4 would miss the 09:15 cab home. This is what happens by default if nothing else is decided.

92%

Night 02, Night 04, Night 06, Night 08

Overtime about 1234

Move the early shift onto the queue

4 early-shift agents already on the floor and trained on this queue. The night shift goes home on time.

92%

Agent 28, Agent 36, Agent 38, Agent 40

Escalate to transport

2 of the affected riders are on Rahul Mikhailov Travel. That is a pattern for the transport manager, not this shift.

Rahul Mikhailov Travel

Billing Support

since 08:25

4 short until 09:19, service level 15%

Billing Support is four agents short until 09:19.

- 3 of 24 on the floor. 16 expected by 09:05, 67% of the roster.
 - Service level per queue, not headcount. 15% and 22% against an 80% target.
 - Day so far still holds 80%. The honest half — this is a floor problem, not yet a contract problem.
 - Every rider's position and arrival, with the ± spread shown, never hidden.
 - The alert reads like a colleague wrote it, with the five benchmarks beneath.
 - Options as buttons, each scored on the contract, recommended one in green.
- ✓ One HTML file, no build step, MoveInSync's palette. Drops into their line-manager desk.

The alert, in the manager's language

Billing Support is four agents short until 09:19. Service level is 15% now, and the day's 80% target still holds. This is tracking towards 5 late, about the usual 5 for this team. Recommend early-shift cover from Agent 27, Agent 29, Agent 35, and Agent 31. If nobody acts, 4 night agents are held 19 min past shift end and 4 would miss the 09:15 cab home.

Verbatim from `GET /alerts` · 2026-06-11 08:55 · `samples/api_alerts_narrated.json`

FACT 1 · SHORTFALL

Rider state machine + arrival projection, guarded by the clock.

COMPUTED

FACT 2 · CONTRACT

Erlang C at 8 agents vs 12, plus the day-level rollup.

COMPUTED

FACT 3 · BENCHMARK

Team norm over 64 days of the same team's history.

COMPUTED

FACT 4 · REMEDY

Cover search over agents with a recorded arrival before now.

COMPUTED

THE PROSE

Which two facts lead, and the five English sentences.

MODEL

The model cannot invent a number or a name — its tools only hand it what was already computed. That is not a prompt instruction. It is a property of the tool surface.

Three options. Each priced, each scored, one click.

Same contract, same axis. The manager compares like with like.

DO NOTHING

92%

Keep the night shift on

Positions stay manned. But 4 night agents are held **29 min** past shift end and 4 miss the 09:15 cab home.

PEOPLE

Night 02, Night 04, Night 06, Night 08

Overtime $\approx$ 1,234

This is what happens by default if nothing else is decided.

RECOMMENDED

92%

Move the early shift onto the queue

4 early-shift agents **already on the floor** and trained on this queue. The night shift goes home on time.

PEOPLE

Agent 27 · Agent 29 · Agent 35 · Agent 31

Hold-over avoided

Each verified by a recorded arrival, and each with zero cover minutes this week — fair as well as fast.

ROUTE ONWARD

Escalate to transport

2 of the affected riders are on **Rahul Mikhailov Travel**. That is a pattern for the transport manager, not this shift.

The agent recognises which problems belong to a different persona and hands them across — rather than asking a line manager to fight a vendor at 08:55.

Every option carries its own cost and its own service-level score, so the choice is a comparison rather than a guess. Clicking one records the decision, returns the draft, and charges the cost to whoever absorbs it.

One click, and the message is already written

RETURNED BY `POST /ALERTS/{ID}/ACT`

To: Billing Support — shift leads

08:55

Billing Support is 67% staffed for the 09:00 start. Could **Agent 27, Agent 29, Agent 35, Agent 31** move onto the queue until the 09:00 team is in? Asking at 08:55.

Verbatim from `samples/api_action.json`. No editing step. No "review before sending" dialog to click through.

AND BEHIND THE CLICK

- ✓ **Decision recorded** in `alert_actions` with the pathway, the people, and the minute — an audit trail a director can query.
- ✓ **Cost charged** to whoever absorbs it — cover pool, night shift, or the contract.
- ✓ **Cover fairness updated** so the same four people are not volunteered every morning.

FAILURE PATH, DESIGNED IN

If the model is down, rate-limited, or out of credits, a **template fallback** still returns a sendable draft for every pathway. The board keeps running. *The act path never depends on an LLM being up.*

Staffing is not linear — and a headcount hides that

Billing Support, real queue parameters. Same offered load, four fewer agents.

12 agents

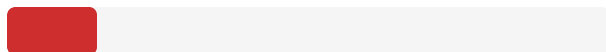
full strength



92%

8 agents

what 08:55 projects



15%

Naive expectation

67% of the people →
67%?



wrong

Losing a third of the agents does not cost a third of the service. It costs **almost all of it**, because occupancy climbs to 96% and the queue stops draining between calls.

AVERAGE WAIT, FULL

8s

AVERAGE WAIT, SHORT

13 min

OCCUPANCY, FULL

64%

OCCUPANCY, SHORT

96%

WHAT THE QUEUE MATHS GIVES US

Erlang C service level, speed of answer, abandonment, occupancy, schedule adherence, a day-level rollup, and `agents_needed` — the last of which is what makes every option comparable on one number.

Honest about the model: Erlang C assumes Poisson arrivals, exponential handle times, no abandonment, and interchangeable agents. Real WFM tools relax all four. The *shape* of the answer is what the decision depends on.

The agent also tells you when *not* to escalate

THE INTERVAL

15%

Answered in 20 seconds, right now, on Billing Support. Against an 80% target. This looks like a five-alarm fire.

THE DAY

92%

Sixteen intervals, zero breached, 736 calls. **The day service level holds the 80% target.** The contract is written on the day, not the interval.

"A twenty-minute gap does not break the day. The interval collapses; the daily number holds. The alert says so, and escalates to operations **only when the day is genuinely at risk.**"

Why this matters more than the alert itself

- An alerting system that fires on every red interval gets **muted within a week**. Then it protects nothing.
- Escalating a recoverable morning burns the manager's credibility with operations, and their own time.
- Restraint is what makes the alert believable on the one morning it does say *escalate now*.

Not escalating saves as much money as the cover recommendation does.

Sense, reason, act — and the model is only in the third box

SENSE

A clock over the trip feed

A replay clock advances one minute at a time over real trip events. **Every read is guarded by now**, so nothing downstream can see the future.

Swap the replay for a live event consumer and nothing else in the system changes. That is the whole production seam.

REASON

Deterministic Python, ~15 ms

Nine-state rider machine → arrival projection with an uncertainty band → queue headcount → Erlang C and the day rollup → cause → five benchmarks → cover candidates → hold-over cost → alert with priced options.

Zero model calls. Zero per-tick round trips.

ACT

The agent writes and answers

A Google ADK agent receives the computed alert and writes the summary and the drafts. The same agent answers the manager's questions from the same computed board.

It cannot invent a number or a name, because the tools only hand it what was computed.

THE TRIGGER DISCIPLINE

The model writes on five events, not on a timer

ALERT OPENS

RESOLVES

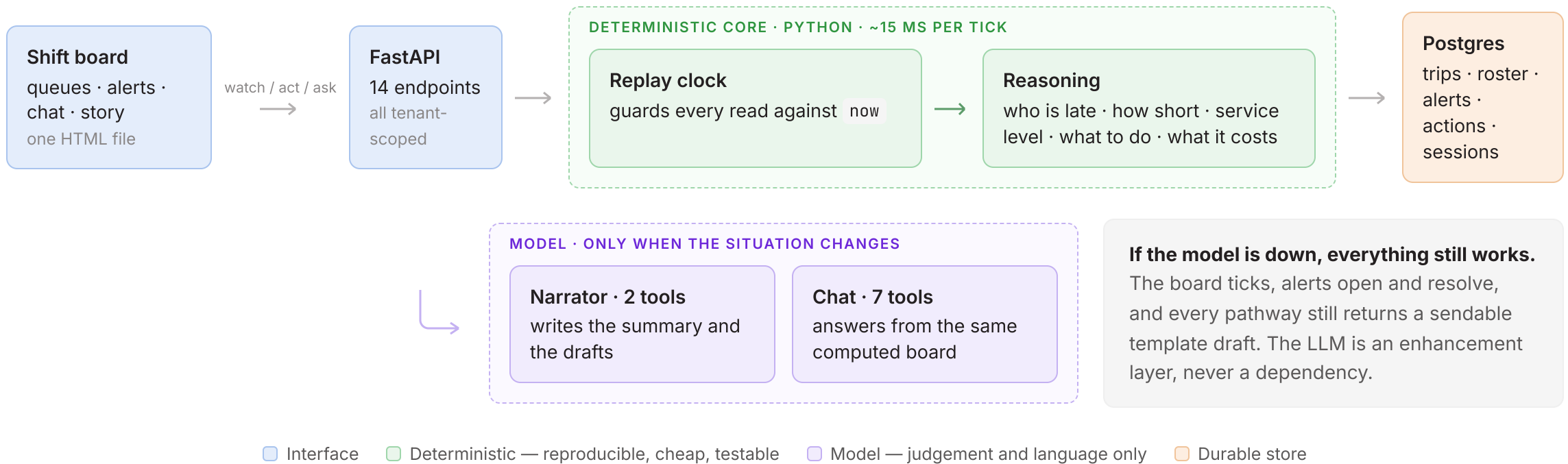
CAUSE CHANGES

RECOMMENDATION CHANGES

SEVERITY BAND CROSSED

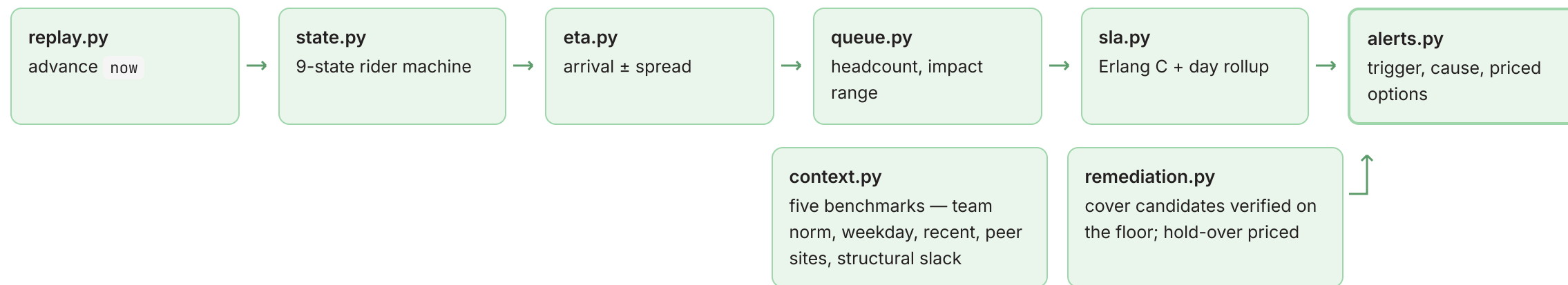
A 150-tick morning across two queues produces **11 narratives**, not 300. Everything else is arithmetic. This one decision is most of the cost story on the next-but-one slide.

Green is arithmetic. Purple is the model.



One tick of the clock, module by module

Each file has one job and its own tests. No model call anywhere in this path.



PER TICK

~15 ms

No model, no per-tick DB round trip. Pooled connections and per-shift caching took the full replay from 18.3 s to 2.29 s.

TESTS

112

Offline, no spend, ~35 s. Plus 3 live tests behind a marker. Each test names the mistake it guards.

ENDPOINTS

14

Every one scoped by business unit, office, date and shift through one Scope dependency.

ROWS LOADED

2.3M

CSV to Postgres via COPY in ~90 s, with nine data quirks handled and each one counted in the load log.

Chosen so the reasoning is the hard part

The brief allows any stack. We optimised for depth of reasoning per hour, and for a clean seam into an existing platform.

DATA & STORE

- **PostgreSQL** — trips, roster, alerts, actions, cover log, narrative cache, *and* agent sessions
- **SQL views** carry the history maths, so benchmarks are a query not a job
- **COPY** bulk load — 2.3M rows in ~90 s
- **psycopg 3** with a connection pool
- **pandas / pyarrow** for the one-off load and quirk reconciliation

REASONING & API

- **Python 3.12**, pure functions, no framework in the core
- **Erlang C** implemented directly — service level, ASA, abandonment, occupancy, adherence
- **FastAPI + uvicorn**, 14 endpoints, one `Scope` dependency for tenancy
- **pytest** — 112 offline tests, 3 live behind a marker
- **uv** for dependency resolution and running

AGENT & INTERFACE

- **Google ADK 2.8** — narrator agent (2 tools) and chat agent (7 tools)
- **LiteLLM** adapter — so the model is a *one-line swap*: OpenAI, Gemini, or local `ollama/llama3.2`
- **ADK DatabaseSessionService** on the same Postgres — one thing to back up
- **One HTML file**, no build step, Inter + Poppins, MoveInSync palette
- **Playwright** for reproducible screenshots

Why not Java and Angular? The brief calls them preferred, not required. In a fixed time box the scarce resource was reasoning depth, not language familiarity — and none of the four evaluation criteria are language-specific. The seam into their platform is an HTTP boundary and a Postgres schema, both of which are language-agnostic.

Where the model is, and where it is deliberately not

COMPUTED IN PYTHON

WRITTEN BY THE MODEL

rider state, arrival, headcount	which two of five facts to lead with
service level, day rollup, adherence	the five sentences of the summary
cause, triggers, option scores	the message to the shift lead
who can cover, what it costs	the answer to a manager's question

The rule: if a number could end up in front of a director, a language model must not have produced it.

Why the numbers must be deterministic

- They get **forwarded to a director**. They must be reproducible on demand.
- Two options are compared on the same axis. If the axis wobbles, the comparison is meaningless.
- They must be identical on a re-run, so a decision can be audited three weeks later.

Why the model earns its place anyway

- Five benchmarks are available; leading with all five is unreadable at 08:55. **Choosing two is judgement**, and it is the model's job.
- A message a manager will actually forward has to read like a person wrote it.
- A follow-up question can take any shape. That is a natural-language problem.

The guarantee is structural, not a prompt. The tools return computed objects. There is no tool that returns raw rows, and scope is not settable by the model. It has nothing to hallucinate *from*.

Metered, not claimed — there is a `/usage` endpoint

Twenty percent of the rubric is inference cost, latency and efficiency at scale. So we measured it.

COST OF ONE TICK

15 ms

And **zero** model calls. The expensive part of the product is free.

NARRATIVES PER MORNING

11

Across 150 ticks and two queues. A naive per-tick agent would make **300**.

TOKENS PER NARRATIVE

~5k

Down from **22,600** in the first working version — a **4.5×** cut.

LATENCY PER NARRATIVE

~9 s

Detached from the clock, so the board never freezes waiting for a model.

How the cost was cut 4.5×

- **Facts moved out of the prompt and into tools.** The model fetches what it needs instead of being handed everything.
- **A throwaway conversation per write-up.** No transcript accumulating across a morning.
- **Narrative cache keyed by payload hash.** The same situation next Tuesday costs nothing.
- **Five change triggers instead of a timer.** This is the single biggest saving.

What this means at enterprise volume

The reasoning core does **no model calls and no per-tick database round trips**. Scaling to a thousand shifts scales a 15 ms pure function, which is an ordinary capacity problem with an ordinary answer.

Model spend scales with **situations that changed**, not with shifts, minutes, or users. A quiet morning at a well-run site costs nothing at all.

That is the difference between an agent you can deploy and a demo you can afford to run once.

Why MoveInSync should want to own this layer

Commute is procured on price and switchable. Floor readiness is not.

1 · DEFEND THE ACCOUNT

Move from a line item to a dependency

Transport is procured, benchmarked and re-tendered on price. The moment a client's line managers start their shift on this board, transport becomes **operational infrastructure** rather than a cost line.

Switching vendor now means retraining every floor manager, not re-running a price comparison.

2 · WIN THE SEGMENT THAT PAYS MOST

BPO and 24/7 enterprise support

These contracts are **won on service level and lost on hold-over and overtime**. They are also the accounts where commute spend is highest and scrutiny is heaviest.

A late cab is their problem only once it becomes an unmanned desk. Nobody currently makes that translation. This does.

3 · NEW VALUE FROM EXISTING DATA

Zero new data collection

Pickup times, drop times, no-shows, vendors — **MoveInSync already has all of it**. This is a reasoning layer on data already being paid for and already flowing.

No new hardware, no new integrations, no client-side instrumentation. The margin on the second sale is very different from the first.

4 · A THIRD SEAT AT THE CLIENT

Reach the line manager, not just the admin

Today MoveInSync's user is the transport admin. This product gives them a daily relationship with **every line manager on every floor** — dozens of users per site instead of one.

Daily-active line managers are the strongest renewal signal an enterprise product can have, and the natural route to a per-seat tier.

What one morning costs, priced by the agent itself

HOLD-OVER COST IF NOBODY ACTS · 11 JUNE 2026 · CLEARWATER CAMPUS

QUEUE	AGENTS HELD	MINUTES	MISSED CABS	COST
Billing Support	4	19 past end	4	799
Technical Support	4	29 past end	4	1,234
One morning, one site	8	—	8	≈ 2,033

FREQUENCY, MEASURED

46%

of 09:00 shifts at this site run late. This is most Thursdays, not a freak morning.

COST OF THE FIX

0

The recommended cover comes from agents *already paid and already seated*. The saving is the avoided overtime, not a trade.

The saving that is probably larger

The rubric's first criterion is *"does it meaningfully reduce manager effort"*. Today that manager spends the twenty minutes before every shift assembling a picture from three systems and a phone. Multiply that by every line manager, every floor, every morning. It never appears on a transport invoice — which is exactly why nobody has tried to remove it.

Second-order effects the board makes visible

- **Abandoned calls.** At 15% service level, modelled abandonment is 87%. Those are customers, not minutes.
- **Night-shift attrition.** Being held past 09:00 and missing the 09:15 cab home, repeatedly, is how good night staff leave.
- **Cover fairness.** Cover minutes are tracked per person per week, so the same four people are not volunteered every morning.

SAID BEFORE YOU ASK

The overtime rate is **synthetic** and lives in `app/config.py`. Substitute the client's real rate and the arithmetic is unchanged. *Not* synthetic: the shortfall, the timing, the people, the service level, and the 46%.

Do not rewrite the system. Swap the clock.

One seam. Everything downstream was written to read only what `now` reveals, so nothing else changes.

MoveInSync live trip / pickup / drop events
already flowing in their platform today

↓ the only replaced component

Event-consuming clock replaces `app/replay.py`
advances `now` from live events instead of recorded legs



Reasoning core — unchanged

`state` → `eta` → `queue` → `sla` → `context` → `remediation` → `alerts`



Agent — unchanged

narrator + chat, model swappable by one env var



Their existing line-manager desk

same JSON the demo board already reads — iframe it, or rebuild the view in Angular against the same API

Why the seam is this clean

Every read in the system is already guarded by `now`. Nothing downstream can see the future, whether `now` comes from a replay or from a Kafka offset. **The guarantee the demo needs is the same guarantee production needs.**

That was not a lucky accident — it is why the clock exists as its own module with its own tests.

What MoveInSync wraps, rather than us building

- **Auth and SSO.** They already have a signed-in desk. We take `business_unit` + `office` on every request.
- **Tenant envelope.** Their existing `x-tenant-id` header pattern maps onto our `Scope` dependency one-to-one.
- **WFM numbers.** Queue handle time, forecast and SLA target come from the client's workforce tool, replacing our synthetic values.
- **Their front end.** Angular against the same JSON, if they would rather not iframe.

Multi-tenancy, latency, and the gaps we chose

PRODUCTION-SHAPED — KEEP

- ✓ **Tenant-scoped throughout.** Every table and all 14 endpoints keyed by `business_unit + office`. Another tenant is two env vars.
- ✓ **~15 ms per tick.** No model on the clock, no per-tick DB round trip.
- ✓ **Durable in Postgres,** including agent sessions. One thing to back up.
- ✓ **Graceful degradation.** Model down → templates. Board never stops.
- ✓ **/health** reports process, mode, and whether a model key is present.

DEMO-ONLY — DO NOT SHIP AS FACT

- **The replay clock.** A recorded morning, not a live consumer. This is the intended seam.
- **The night shift and handover.** Asserted, because this site's log has no outbound legs near 09:00. Flagged `synthetic` on the roster row.
- **Queue AHT, call forecast, SLA target, overtime rate.** Synthetic. Replaced by the client's WFM numbers.
- **The story slider.** A presenter tool, not the product. It is `app/timeline.py` and it is clearly separated.

DELIBERATELY NOT BUILT

- **Auth, SSO, API keys.** The brief explicitly does not expect them, and MoveInSync already has a signed-in desk.
- **Multi-region, sharding, an event bus.** Their production plan already talks CDC and Kafka for trip events; that feed *is* the replacement clock.
- **GPS.** Not in the dataset. See the limitations slide — we handle its absence rather than pretending.
- **The other two personas.** Building one persona properly beats three shallowly, and the rubric rewards landing with the intended user.

Runs on Render, Vercel, or Cloud Run. The ADK agent deploys with one command.

What is real, and what we invented

Every synthetic value is labelled in the code and in the roster, not only on this slide.

REAL — FROM THE DATASET

- Riders, trips, pickup and drop times, no-shows, vendors, cabs
- **The 24-person team** — real riders, selected because they rode this shift on 40+ days
- **The 24-person cover pool** — same, on the 08:30 shift
- All five benchmarks: team norm over 64 days, weekday pattern over 12 Thursdays, recent trend, four peer sites, structural slack over 1,413 legs
- The 46% late rate, the 70% `NODELAY` rate, the 5-vs-13-minute buffer finding
- Every arrival time, every projection, every shortfall on the board

SYNTHETIC — AND FLAGGED IN CODE

- Which queue a rider serves, handle time, call forecast, service target — in the `roster` and `queues` tables
- Overtime rate, cab-home time, cover fairness counter — in `app/config.py`
- **The night shift and positional handover** — asserted, and marked `synthetic` on the roster row

Why we invented the night shift, specifically. Clearwater's trip log has no outbound legs anywhere near 09:00, so there was no night shift in the data to read. We asserted one because *positional handover* — a night agent cannot leave until their relief is seated — is what makes a late arrival cost somebody their morning, rather than merely thinning a queue. Remove it and the product is a headcount widget.

The rule we held to: never let a synthetic number do work that a real one could do. The night shift is the single invention that changes the shape of the product, and it is the one we flag loudest.

What we cannot do, and what we did about it

LIMITATION

No GPS in the dataset

Pickup and drop times only. Once a rider is aboard, nothing is observable until they arrive — **~13 minutes of irreducible spread**.

WHAT WE DID

Impact is reported as a **range**, never a false point estimate, and the $\pm$ spread shows on every arrival. The hard trigger is the deterministic *"planned drop passed, no arrival"* — no false positives, with or without GPS.

LIMITATION

The projection is optimistic on bad days

It is calibrated on ordinary mornings. On genuinely bad ones it under-predicts lateness.

WHAT WE DID

This is **asserted in a test**, so nobody who inherits the code mistakes a known bias for a bug — and so it fails loudly if someone later "fixes" it in the wrong direction.

LIMITATION

Erlang C makes four assumptions

Poisson arrivals, exponential handle times, no abandonment, interchangeable agents. Real WFM tools relax all four.

WHAT WE DID

Used it anyway, because **the shape of the answer is what the decision depends on** — 92% to 15% is the insight, not the third significant figure. It lives in one module, so a client's WFM engine can replace it wholesale.

LIMITATION

The night shift is invented

The most load-bearing synthetic element in the build. See the previous slide.

WHAT WE DID

Labelled everywhere — config, roster row, README, and a slide before anyone asks. **Hold-over minutes are not quoted as measured fact** until a real logout roster exists.

Mapped to the four published criteria

WT.	CRITERION	OUR EVIDENCE
35	Business impact & experience Reduces manager effort; surfaces missed decisions; clarity for the persona; leadership-ready and shareable without rework.	One persona, served completely. Twenty minutes of morning assembly reduced to one screen and one click. Every alert is a forwardable paragraph with five benchmarks behind it, and the draft message needs no editing. The agent surfaces a decision the manager would otherwise miss entirely — 70% of these late arrivals are invisible to the operator's own delay metric . Hold-over is priced, and it also says when <i>not</i> to escalate.
20	Agentic design & cost at scale Genuine problem, not decoration; inference cost, latency, efficiency at enterprise volume.	Senses on a guarded clock, reasons in 15 ms of pure Python, and acts with a recorded decision and a sent message. The model is confined to judgement and language, and fires on five change events, not a timer — 11 narratives per morning instead of 300. Tokens cut 4.5× from the first working version. A live <code>/usage</code> meter, and a cache that makes a repeat situation free.
20	Architecture & code quality Sound structure; deployable into an existing platform; choices a team could build on.	One seam between demo and production: replace the clock. Every table and endpoint tenant-scoped through a single dependency. Modules with one job each and 112 tests that name the mistake they guard against . The model is a one-line swap. Failure paths designed in, not bolted on.
25	Functionality It runs — working, demo-able, end to end, on the provided dataset.	2.3M rows loaded from the provided CSVs with nine documented quirks handled. A full morning replays with alerts opening, narrating, being acted on, and resolving. Chat streams. Live demo next — plus a precomputed story mode so a slow network can never cost us the demo.

Deliverables, all present: source repository · architecture diagram (SVG, PNG, Excalidraw, Mermaid) · README with setup · sample inputs and outputs in `samples/` · this deck · live demo.

One real morning, in six beats

11 June 2026 · Clearwater Campus · the 09:00 shift. Landmarks are read off the event feed, not hand-placed.

07:45

GREEN

Board is green. 24 rostered, nobody due yet.
This is what a good morning looks like.

08:12

FIRST AMBER

The cab for Agent 20 has not left its depot, 32 minutes late. Sensed before any human noticed.

08:25

ALERT OPENS

Billing Support alert opens at 92% projected coverage. The written narrative arrives seconds later.

08:47

ESCALATING

Agent 15 not collected, the cab has passed.
The alert now offers "Call the unaccounted riders".

08:55

THE CRUNCH

Billing at 67%, service level 15%. Click the green option. The draft appears under Sent, naming four people verifiably on the floor.

09:19

RESOLVES

Billing back to 83%. The alert resolves itself.
Then ask chat: *"Who covered this morning, and how much overtime did we avoid?"*

The story slider is precomputed — and that is not a trick

Every minute from 07:30 to 10:00 is captured, with the landmarks derived from the event feed. **Every snapshot is the same deterministic function the live clock runs**, evaluated at that minute, with the same guard against reading the future. The narratives are real model output, produced in order, so the write-up at 08:25 is the one that was written at 08:25.

WHY IT EXISTS

So that a slow network, a rate limit, or an empty API balance can never cost us a demo. **"Back to live" returns to the running clock at any moment.**

MoveInSync already sees the commute. Nobody was translating it for the floor.

One real morning, on their own trip log. At 08:55 Billing is four short, service level 15%, and the night shift is about to be held. **The numbers are arithmetic; the model writes the sentence and the draft.** One click moves four people already on the floor — message ready, overtime priced. Swap the replay for a live trip feed and the same API sits in the desk they already have.

PERSONA

One line manager, served completely.

AGENTIC

Senses, reasons, acts. 11 narratives a morning.

ARCHITECTURE

One seam to production. 112 tests.

IT RUNS

2.3M rows. End to end. Live, next.

TEAM

NEON

REPOSITORY

README · SHIP.md ·
JURY.md

SAMPLES

samples/ — every figure in
this deck

RUN IT

uv sync · uvicorn
app.api:app